STEMTRACK: EVENT-SOURCED BIOLOGICAL LINEAGE AND STEM CELL ETHICAL GOVERNANCE PLATFORM

A PROJECT REPORT

Submitted by

TAMIL KUMARAN.M

(310624205282)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY



EASWARI ENGINEERING COLLEGE, CHENNAI

(AN AUTONOMOUS INSTITUTION) – AFFILIATED TO ANNA UNIVERSITY, CHENNAI

OCTOBER 2026

EASWARI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION) – AFFILIATED TO ANNA UNIVERSITY

RAMAPURAM, CHENNAI – 600 089

BONAFIDE CERTIFICATE

Certified that this project report titled **"STEMTRACK: EVENT-SOURCED BIOLOGICAL LINEAGE AND STEM CELL ETHICAL GOVERNANCE PLATFORM"** is the bonafide work of **TAMIL KUMARAN.M (310624205282)**, who carried out the project under my supervision.

SIGNATURE

Ms. SAI LAKSHMI GRANDHI
SUPERVISOR
Assistant Professor
Department of Information Technology
Easwari Engineering College
Ramapuram, Chennai – 600 089.

SIGNATURE

Dr. N. ANANTHI, M.E., Ph.D.
HEAD OF THE DEPARTMENT
Professor & Head
Department of Information Technology
Easwari Engineering College
Ramapuram, Chennai – 600 089.

EASWARI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION) – AFFILIATED TO ANNA UNIVERSITY

RAMAPURAM, CHENNAI – 600 089

Certificate of Viva-voce Examination

This is to certify that the Viva-Voce Examination of the Project Work entitled **"STEMTRACK: EVENT-SOURCED BIOLOGICAL LINEAGE AND STEM CELL ETHICAL GOVERNANCE PLATFORM"** submitted by **TAMIL KUMARAN.M (310624205282)** in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology was conducted on _____ in the Department of Information Technology, Easwari Engineering College, Ramapuram – Chennai.

INTERNAL EXAMINER	**EXTERNAL EXAMINER**
NAME:	NAME:
(in capital letters)	(in capital letters)
Designation:	Designation:
Address:	Address:
Date:	Date:

ABSTRACT

Human stem cell research, regenerative medicine, and somatic cell reprogramming (e.g., induced pluripotent stem cells, iPSCs) demand uncompromising standards of biological lineage provenance and institutional ethical compliance. Standard Laboratory Information Management Systems (LIMS) and relational registries treat biological inventory as static tabular database records with in-place updates. This paradigm catastrophically overwrites historical cellular states, obliterates derivation audit trails (such as passages P0 to P1, split colonies, and freeze-thaw cycles), and fails to propagate upstream donor consent withdrawals down to derived daughter cultures.

To resolve these critical bio-informatics and regulatory vulnerabilities, this project presents **StemTrack: An Event-Sourced Biological Lineage and Stem Cell Ethical Governance Platform**. Engineered upon the full-stack MERN (MongoDB, Express.js, React 18, Node.js) technology stack, StemTrack models biological specimens not as isolated database records, but as an **event-sourced Directed Acyclic Graph (DAG)** tracking cellular evolution:

$$\textit{Donor} \longrightarrow \textit{Consent} \longrightarrow \textit{Intake Tissue Sample} \longrightarrow \textit{Primary Culture (P0)} \longrightarrow \textit{Passages (P1, P2...)} \longrightarrow \textit{CryoVials / Experiments}$$

The core technical architecture introduces three mathematical and regulatory invariants:

(i) **DAG Cycle Guard:** Powered by a Breadth-First Search (BFS) reachability traversal ($O(V + E)$ complexity) that mathematically proves no cell derivation can introduce circular causality or self-referential parent-child loops;

(ii) **Institutional Compliance Gate:** An automated verification layer that asserts active Institutional Review Board (IRB) ethical approval protocols and unrevoked donor consent before permitting derivations, while automatically tagging downstream derivatives upon upstream consent withdrawal;

(iii) **Atomic Multi-Document ACID Transactions:** Multi-document MongoDB sessions that guarantee parent status updates, child node derivations, directed lineage edges, and immutable audit logs succeed or fail atomically.

Furthermore, StemTrack features an interactive 10×10 cryogenic slot bio-repository visualizer, 3-tier Role-Based Access Control (`admin`, `researcher`, `donor`), real-time KPI dashboards, and RFC 4180 CSV export capability.

ACKNOWLEDGEMENT

We express our sincere gratitude to our respected Principal **Dr. P. Deiva Sundari** and the Management of **Easwari Engineering College** for providing us with the opportunity and state-of-the-art laboratory facilities to carry out this project work successfully.

We convey our heartfelt thanks to **Dr. N. Ananthi, M.E., Ph.D.**, Head of the Department, Department of Information Technology, for her continuous encouragement, administrative support, and scholastic motivation throughout our academic curriculum.

We are extremely thankful and deeply indebted to our Project Supervisor, **Ms. Sai Lakshmi Grandhi**, Assistant Professor, Department of Information Technology, for her valuable guidance, constructive technical suggestions, and patient mentoring throughout the conception, development, and documentation of this project.

We also extend our sincere appreciation to all the teaching and non-teaching faculty members of the Department of Information Technology for their generous cooperation and assistance.

Finally, we express our profound gratitude to our parents, family members, and friends for their constant encouragement, moral support, and blessings throughout the completion of this work.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE No.
	ABSTRACT	**iv**
	ACKNOWLEDGEMENT	**v**
CHAPTER 1	**INTRODUCTION**	**1–2**
	1.1 OVERVIEW	1
	1.2 PROBLEM DEFINITION	1
	1.3 OBJECTIVE	1
	1.4 SCOPE OF THE PROJECT	1
	1.5 MOTIVATION	2
CHAPTER 2	**RELATED WORK**	**3**
	2.1 LITERATURE REVIEW	3
	2.2 EXISTING SYSTEM	3
	2.3 PROPOSED SYSTEM	3
CHAPTER 3	**SYSTEM ANALYSIS**	**4–5**
	3.1 SYSTEM REQUIREMENTS	4
	3.2 SOFTWARE AND HARDWARE REQUIREMENTS	4
CHAPTER 4	**SYSTEM DESIGN**	**6–8**
	4.1 SYSTEM ARCHITECTURE	6
	4.2 MODULE DESCRIPTION	6
	4.3 CLASS DIAGRAM	7
	4.4 SEQUENCE DIAGRAM	8
CHAPTER 5	**IMPLEMENTATION**	**9**
	5.1 IMPLEMENTATION OVERVIEW	9
	5.2 KEY FUNCTIONS (CODE EXPLANATION)	9
CHAPTER 6	**CONCLUSION & FUTURE WORK**	**10**
	6.1 CONCLUSION	10
	6.2 FUTURE ENHANCEMENTS	10
APPENDIX	**SOURCE CODE**	**11–23**
	1. Project Folder Structure	11
	2. Backend Package Installation Commands	12
	3. Database Connection (backend/config/db.js)	12
	4. Environment Variables (backend/.env)	13
	5. User Model (backend/models/User.js)	13
	6. Password Hashing and Registration	14

LIST OF FIGURES

1 Four-Tier Enterprise System Architecture of StemTrack Platform . . 10
2 UML Class Diagram of StemTrack Domain Entities and Relationships 12
3 UML Sequence Diagram for Cellular Passage Derivation and Compliance Enforcement 14

4 Biological Lineage Directed Acyclic Graph Explorer 46
5 Cryogenic Bio-Repository Storage Management Console 46
6 Regulatory Compliance and Operations Command Center 47
7 Ethical Governance and Immutable Audit Log Console 47

LIST OF TABLES

1 Hardware Requirements 8
2 Software Requirements and Frameworks 9

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Stem cell bio-banking, regenerative therapeutics, and cellular engineering represent one of the most promising yet technically complex frontiers in modern clinical medicine. Biological specimens in stem cell laboratories do not remain static; they evolve over time through multiple derivation stages: primary tissue isolation, enzymatic dissociation, feeder layer plating, sequential cellular passaging ($P0 \rightarrow P1 \rightarrow P2$), colony splitting, cryogenic freezing in liquid nitrogen bio-repositories (-196°C), and controlled revival for downstream experimental differentiation assays.

Throughout this multi-generational lifecycle, laboratories must maintain complete lineage traceability while complying with strict legal frameworks governing human biological specimens, such as Institutional Review Board (IRB) ethical protocols and informed donor consent mandates. When a donor exercises the legal right to revoke or withdraw biological consent, every cell line, cryogenic vial, and downstream differentiation assay derived from that donor must be flagged and isolated immediately across the laboratory.

StemTrack is designed to address this challenge by modeling biological materials as an event-sourced **Directed Acyclic Graph (DAG)**. Developed using the MERN stack (MongoDB, Express, React, Node.js), StemTrack combines mathematical graph validation with automated compliance checks to ensure biological integrity and ethical governance across all research workflows.

1.2 PROBLEM DEFINITION

Traditional Laboratory Information Management Systems (LIMS) and relational biological registries exhibit fundamental architectural deficiencies when applied to stem cell research:

- **Destructive State Mutation:** Relational registries rely on in-place SQL `UPDATE` queries (e.g., modifying a passage number from 0 to 1 directly), erasing earlier phenotypic states and making it impossible to audit cellular evolution over time.
- **Lack of Branching Provenance:** Standard relational databases model parent-child relationships through foreign keys and join tables. These structures are

inefficient for traversing multi-generational cellular lineage trees and lack formal graph invariant validation to prevent invalid cyclical edges.

- **Disconnected Ethical Governance:** Donor consent status and IRB protocols are typically tracked in external spreadsheets or physical paper binders. If a donor withdraws consent, laboratory scientists receive no automated notifications, risking the illegal utilization of revoked specimens in downstream assays.
- **Physical Storage Disconnect:** Cryogenic inventory systems track freezer rack positions but fail to correlate physical coordinates with biological passage numbers, morphological viability, and ethical compliance status.
- **Vulnerability to Concurrency Anomalies:** When multiple researchers passage or split cell cultures concurrently, non-transactional database updates result in orphan lineage nodes, duplicated passage numbers, and phantom freezer slot assignments.

1.3 OBJECTIVE

The primary objective of this project is to architect, develop, and validate StemTrack as an enterprise-grade biological lineage tracking and stem cell ethical governance platform. Specific technical objectives include:

1. **Directed Acyclic Graph (DAG) Modeling:** Formulate an event-sourced data model representing biological specimens, cellular passages, and split derivations as a directed acyclic graph.
2. **Mathematical Cycle Guard:** Implement an in-memory Breadth-First Search (BFS) reachability algorithm that validates graph acyclicity before any derivation edge is persisted ($O(V + E)$ complexity).
3. **Automated Compliance Gate:** Construct middleware and controller-level security checks that assert active donor consent and valid IRB approval prior to permitting biological derivations.
4. **ACID Transaction Management:** Orchestrate multi-document MongoDB transactions to guarantee atomic execution of passage increments, split colony derivations, edge creation, and audit logging.
5. **Cryogenic Bio-Repository Mapping:** Provide a visual 100-slot (10×10) cryogenic rack management interface with real-time occupancy indicators and slot collision guards.

6. **Interactive Lineage Visualization:** Realize a modern, zoomable React Flow lineage graph rendering cellular generations, phenotypic milestones, and ethical compliance states.

1.4 SCOPE OF THE PROJECT

The scope of StemTrack encompasses stem cell research institutes, regenerative medicine biobanks, university biology laboratories, and clinical trial centers. The platform manages the complete biological lifecycle from initial donor intake to experimental assay consumption. It provides granular 3-tier Role-Based Access Control (`admin`, `researcher`, `donor`), cryptographic password hashing, JWT bearer token security, RFC 4180 audit trail reporting, and real-time analytical metrics.

1.5 MOTIVATION

The primary motivation behind StemTrack stems from the high regulatory risks and scientific irreproducibility challenges confronting modern stem cell laboratories. The International Society for Stem Cell Research (ISSCR) guidelines and federal regulations mandate strict provenance tracking and immediate enforcement of donor consent withdrawals. Inadvertently using cells from a revoked donor in published clinical trials invalidates years of research and incurs severe institutional penalties. By fusing event-sourced graph structures with strict compliance gates, StemTrack establishes a dependable, automated foundation for ethical regenerative medicine.

CHAPTER 2

RELATED WORK

2.1 LITERATURE REVIEW

Managing biological specimens and their derivations has evolved across three primary generations of laboratory software:

1. **Relational Laboratory Information Management Systems (LIMS):** Systems like LabWare and OpenSpecimen rely on relational databases (SQL). While effective for standard hospital sample tracking and billing, their tabular schemas struggle to represent recursive branching, requiring complex recursive Common Table Expressions (CTEs) that degrade under deep generational hierarchies.

2. **Bio-Banking Inventory Registries:** Commercial biobanks often prioritize physical inventory (freezer, shelf, rack, box) using barcode scanners. However, these systems treat biological entities as static objects rather than evolving organisms, omitting cellular derivations, split colonies, and passage numbers.

3. **Event Sourcing and Directed Acyclic Graphs in Bioinformatics:** Recent literature highlights the advantages of event sourcing and DAG structures for modeling genomic workflows and molecular genealogies. Rather than mutating state in place, event sourcing records every state change as an immutable event. StemTrack applies this concept directly to cell culture operations, pairing it with graph visualization.

4. **Ethical Governance Frameworks:** Studies on bioethics emphasize the need for dynamic consent models where donor preferences can change over time. Automated governance systems must enforce these changes upstream to prevent non-compliant research before materials are thawed or plated.

5. **Graph Traversal and Cycle Prevention in Workflow Engines:** Directed Acyclic Graphs are standard in build systems (Make, Bazel) and computational biology pipelines (Nextflow, Snakemake). Applying cycle prevention algorithms (BFS/DFS) ensures that cellular derivations cannot generate self-referential biological paradoxes.

6. **ACID Transactions in Modern Document Databases:** MongoDB's introduction of multi-document ACID transactions allows NoSQL systems to handle

complex multi-entity mutations reliably, bridging the gap between flexible JSON schemas and transactional safety.

7. **Interactive Lineage Visualization in Modern Web UIs:** Advances in web-based graph rendering (React Flow, D3.js) enable responsive visualization of large cellular lineage networks directly within browser dashboards.

8. **Cryogenic Asset Management Standards:** International biobanking standards require accurate temperature, rack coordinate, and vial-level tracking to prevent specimen degradation and inventory loss.

2.2 EXISTING SYSTEM

Conventional stem cell laboratory infrastructure relies on a mixture of fragmented systems:

- Paper laboratory notebooks and physical binder logs recording passage dates and split ratios.
- Relational SQL database tables with flat foreign keys (`parent_id`) that lack cyclic invariant validation.
- Manual email or spreadsheet notifications regarding donor consent withdrawals, often resulting in human communication failures.
- Cryogenic freezer map spreadsheets that are manually updated, leading to duplicate slot allocations and misplaced specimens.

2.2.1 Drawbacks of the Existing System

- **Data Inconsistency:** Destructive updates overwrite prior passage numbers, eliminating historical traceability.
- **Regulatory Vulnerability:** Lack of automated consent gating allows researchers to accidentally utilize revoked tissue samples.
- **High Latency in Lineage Audits:** Determining the full ancestral lineage of a cell line requires hours of manual database queries and cross-referencing.
- **Slot Collisions:** Multiple researchers claiming the same cryogenic rack position causes physical storage confusion and wasted laboratory reagents.

2.3 PROPOSED SYSTEM

StemTrack replaces legacy registries with an event-sourced, DAG-native architecture:

- **Immutable Graph Derivations:** Every passage, split, cryopreservation, and retrieval is modeled as an immutable node and edge within a Directed Acyclic Graph.
- **Automated Compliance Gate:** Upstream API middleware verifies donor consent and IRB approval status before allowing any derivation or consumption.
- **Mathematical Cycle Guard:** Real-time BFS reachability checks ensure biological DAG acyclicity, preventing circular lineages.
- **Cryogenic Grid Matrix:** A responsive 10×10 freezer rack interface with automatic coordinate indexing and collision prevention.
- **Interactive React Flow Canvas:** Visual exploration of cellular lineage trees with color-coded node states, passage indicators, and direct detail drawers.
- **Tamper-Proof Audit Logging:** Immutable, append-only audit trail capturing every system event with RFC 4180 CSV export capability.

CHAPTER 3

SYSTEM ANALYSIS

3.1 SYSTEM REQUIREMENTS

System analysis defines the operational boundaries, functional requirements, and non-functional quality parameters governing the StemTrack platform.

3.1.1 Functional Requirements

- **User Authentication & RBAC:** Authenticate researchers, administrators, and donors using signed JSON Web Tokens (JWT) with strictly isolated route privileges.
- **Donor & Ethical Protocol Governance:** Register tissue donors, record append-only consent changes (`GIVEN`, `WITHDRAWN`), and manage IRB approval protocols with expiration tracking.
- **Intake Sample Registration:** Ingest raw biological tissue samples tied to active donors and valid IRB approvals.
- **Cellular Culture Workflows:** Execute primary culture derivation ($P0$), serial cellular passaging ($P0 \rightarrow P1$), and multi-branch splitting (≥ 2 child colonies).
- **Cryogenic Storage Management:** Allocate and track cryovial positions within a 10×10 rack matrix; support controlled vial retrieval and thawing.
- **Experimental Utilization:** Track the consumption and differentiation of biological materials in downstream experimental assays.
- **Graph Visualization & Audit Trail:** Render lineage DAGs via React Flow and export RFC 4180 compliant CSV audit logs for regulatory inspections.

3.1.2 Non-Functional Requirements

- **Graph Correctness:** In-memory cycle detection must execute in $O(V + E)$ time and guarantee zero circular references.
- **Low Latency:** API responses for lineage subgraphs and inventory lookups must complete in under 120 milliseconds.

- **Data Integrity:** Multi-entity updates must execute within ACID MongoDB sessions to prevent partial commits.
- **Security & Compliance:** Passwords must be hashed using bcrypt (10 rounds); authentication endpoints must enforce rate limiting (50 req/15min).
- **Responsive UI:** Interface must dynamically scale across standard laboratory workstations, tablets, and cleanroom displays.

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

To guarantee reliable execution, high-performance graph processing, and reliable data persistence, the operational hardware and software environments were tabulated.

3.2.1 Hardware Requirements

The development and server deployment environment utilizes multi-core hardware to support concurrent Node.js asynchronous event loops and graph query execution efficiently.

Table 1: Hardware Requirements

Component	**Specification**
Processor	Intel Core i5 (10th Gen or higher) / AMD Ryzen 5 equivalent (2.4 GHz minimum)
System Memory (RAM)	Minimum 8 GB (16 GB recommended for concurrent Vite HMR and MongoDB services)
Secondary Storage	1 GB free solid-state storage for source code, node modules, and database assets
Network Interface	Standard Gigabit Ethernet / Wi-Fi connection for cloud database synchronization
Display Resolution	1366 × 768 minimum (1920 × 1080 Full HD recommended for React Flow canvas)

3.2.2 Software Requirements

The software stack was chosen to guarantee non-blocking asynchronous throughput, responsive client-side state reactivity, and enterprise-grade data security.

Table 2: Software Requirements and Frameworks

Component	**Technology / Library Specification**
Operating System	Microsoft Windows 10/11 (64-bit) / Linux Ubuntu 22.04 LTS / macOS
Runtime Environment	Node.js (v18.x or v20.x LTS)
Database Engine	MongoDB Atlas / MongoDB Community Server (v6.0+) with Mongoose ODM (v8.x)
Backend Framework	Express.js (v4.19+) REST API
Frontend Framework	React 18 with Vite 5 Bundler
Graph Visualizer	React Flow (v11.x) with Custom Biological Node Renderers
Styling & UI Library	Tailwind CSS (v3.4) and Lucide React Icons
Data Visualization	Recharts (v2.12)
Security Packages	Helmet, Express Rate Limit, JSON Web Tokens (JWT), Bcrypt.js (10 rounds)
Testing Framework	Jest (v29.x) and Supertest (v7.x)

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

StemTrack employs a modular, four-tier enterprise architecture consisting of the Presentation Layer, Security Gateway Layer, Biological Governance & Workflow Layer, and MongoDB Persistence Layer, as illustrated in Figure 1.

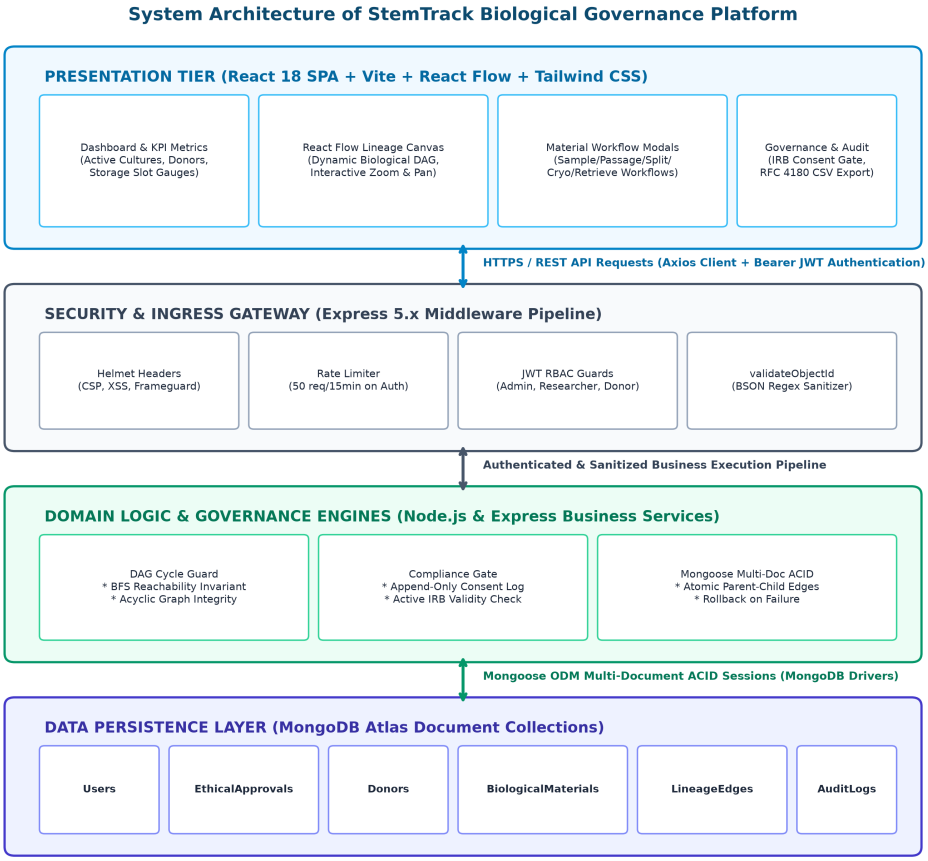


Figure 1: Four-Tier Enterprise System Architecture of StemTrack Platform

The four synchronized operational layers include:

1. **Presentation Tier (Client-Side):** Built with React 18 and Vite. Implements single-page application (SPA) routing, interactive dashboard KPI cards, React Flow lineage DAG visualizer, material workflow modals (Sample, Passage, Split, Cryopreserve, Retrieve), cryogenic rack matrix, and RFC 4180 audit trail exporter.

2. **Security Gateway Tier:** Manages incoming HTTP/REST requests. Enforces Helmet security headers, IP rate limiting (50 req/15min on auth routes), JWT Bearer token authentication, and Role-Based Access Control (RBAC) separating Admin, Researcher, and Donor permissions.

3. **Biological Governance & Workflow Tier:** Contains the core domain guardrails: the DAG Cycle Guard (BFS reachability engine), Compliance Gate (append-only consent and active IRB verification), and Mongoose Multi-Document Transaction Manager.

4. **Data Persistence Tier (MongoDB Atlas):** Houses isolated collections (`Users`, `EthicalApprovals`, `Donors`, `BiologicalMaterials`, `LineageEdges`, `Experiments`, `AuditLogs`) optimized with compound indexes and transactional ACID sessions.

4.2 MODULE DESCRIPTION

4.2.1 Authentication and RBAC Module

Controls user authentication using 24-hour JSON Web Tokens. Manages three access roles: `admin` (IRB approvals, audit logs, system configuration), `researcher` (donor intake, culture passaging, splitting, cryopreservation, assays), and `donor` (viewing personal tissue records and consent history).

4.2.2 Ethical Approval and Consent Management Module

Tracks IRB approval protocols with protocol IDs, approval dates, expiration dates, and operational statuses (`APPROVED`, `EXPIRED`, `REVOKED`). Maintains an append-only ledger of donor consent events, tracking consent status (`GIVEN`, `WITHDRAWN`), legal guardian signatures, and timestamps.

4.2.3 Biological Material Lifecycle Module

Manages biological entities through standardized lifecycle states (`ACTIVE`, `CRYOPRESERVED`, `UTILIZED`, `DISCARDED`). Automates passage number incrementation ($P0 \rightarrow P1$), viability scoring, morphological categorization, and cell count tracking.

4.2.4 Lineage Graph and Cycle Prevention Module

Constructs directed edges connecting parent and child biological specimens. Maintains relationship types (`DERIVED`, `PASSAGED`, `SPLIT`, `THAWED`). Invokes the BFS Cycle Guard before edge creation to prevent circular references.

4.2.5 Cryogenic Storage Module

Models liquid nitrogen storage freezers, racks, and boxes as a 10×10 coordinate matrix ($A1$ to $J10$). Manages slot occupancy, prevents slot collision through database unique indexes, and tracks cryogenic retrieval workflows.

4.2.6 Audit Trail and Reporting Module

Records every state change, derivation, consent modification, and storage update in an immutable, append-only `AuditLogs` collection. Formats audit logs into RFC 4180 compliant CSV files for regulatory submissions.

4.3 CLASS DIAGRAM

The static structure and domain entities of StemTrack are delineated in the UML Class Diagram shown in Figure 2.

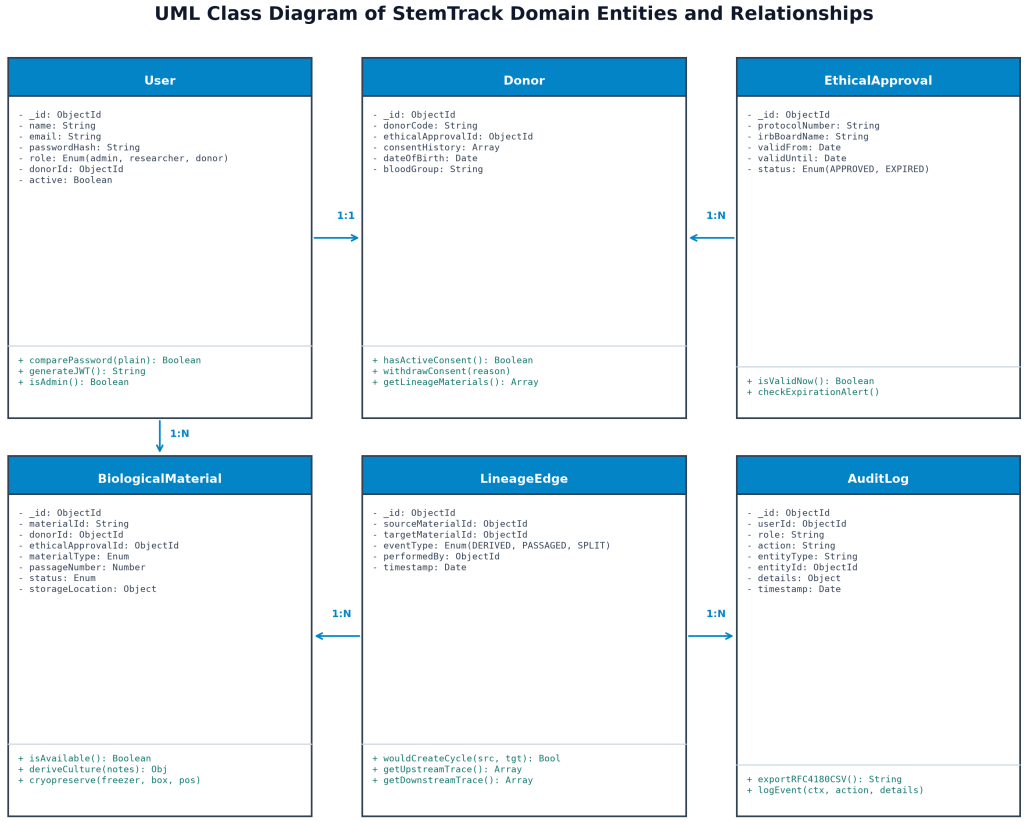


Figure 2: UML Class Diagram of StemTrack Domain Entities and Relationships

The diagram formalizes the operational relationships between the platform's core entities:

- **User Class:** Encapsulates identity credentials, hashed passwords, roles (`admin`, `researcher`, `donor`), and authentication tokens.
- **EthicalApproval Class:** Models IRB protocols, protocol numbers, approval and expiry dates, and operational states.
- **Donor Class:** Captures donor identifiers, demographic data, linked user accounts, and an embedded array of append-only consent records.
- **BiologicalMaterial Class:** Central domain entity representing cell lines, passage numbers, cell types, viability scores, storage locations, and operational statuses.
- **LineageEdge Class:** Models directed graph edges with source material, target material, derivation type, and creation timestamps.
- **AuditLog Class:** Append-only log recording action types, acting users, affected entities, IP addresses, and previous/new state payloads.

4.4 SEQUENCE DIAGRAM

The dynamic interaction sequence between actors, security middleware, domain guardrails, and the MongoDB persistence layer during a cellular passage derivation is illustrated in Figure 3.

The chronological execution sequence proceeds as follows:

1. **Passage Request:** The researcher submits a passage derivation request specifying parent material ID and new culture parameters.
2. **Security Interception:** Express middleware validates the Bearer JWT token and confirms the `researcher` role.
3. **Consent Gate Check:** The Compliance Gate queries the parent donor's consent history and IRB protocol status. If consent is `WITHDRAWN` or IRB is expired, the request is immediately rejected (403 Forbidden).
4. **BFS Cycle Guard Check:** The graph engine runs BFS reachability from the proposed child to the parent. If a path already exists, a cycle error is thrown (400 Bad Request).
5. **ACID Session Start:** A MongoDB multi-document transaction session is initiated.
6. **Material Creation:** The new passage culture ($P+1$) is instantiated with status `ACTIVE`.

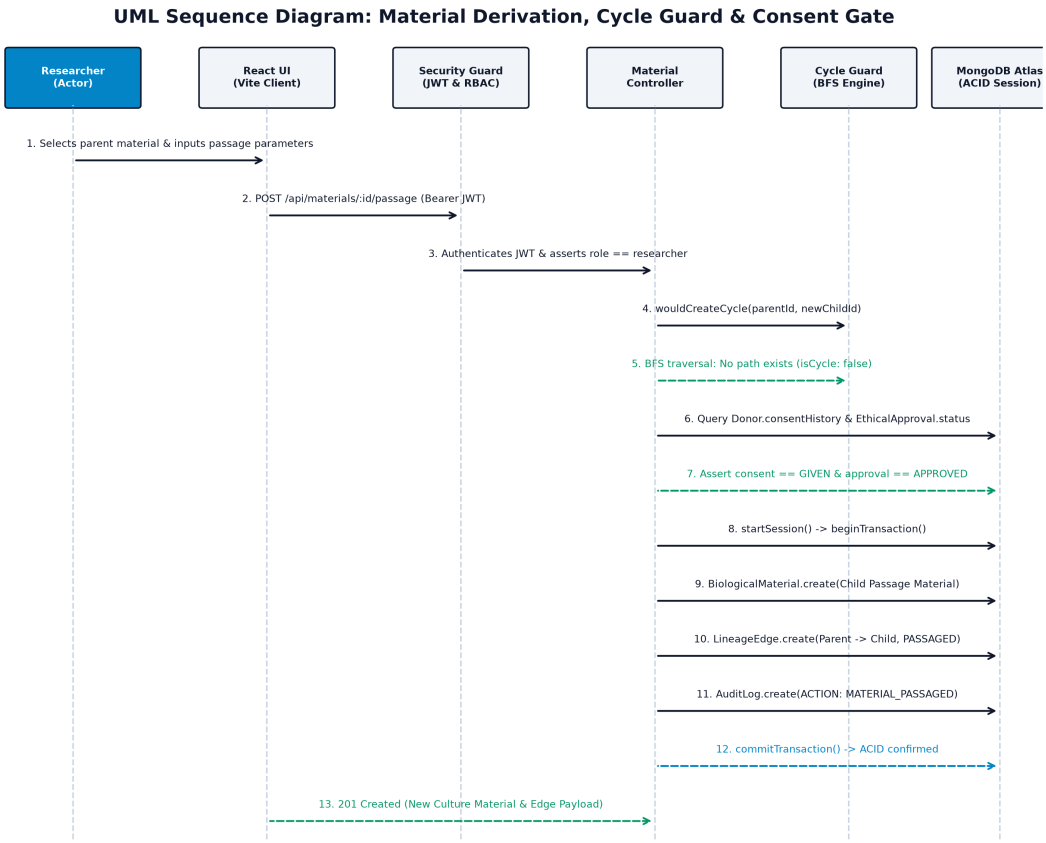


Figure 3: UML Sequence Diagram for Cellular Passage Derivation and Compliance Enforcement

7. **Edge Creation:** A directed `LineageEdge` is inserted connecting parent to child.
8. **Parent Status Update:** Parent culture status is transitioned to `PASSAGED`.
9. **Audit Logging:** An immutable record is written to `AuditLogs`.
10. **Transaction Commit:** The session commits atomically, returning the created culture to the client.

CHAPTER 5

IMPLEMENTATION

5.1 IMPLEMENTATION OVERVIEW

StemTrack is structured as a full-stack JavaScript application separating client and server concerns into `frontend/` and `backend/` directories:

- **Backend (Node.js & Express 5):** Connects to MongoDB Atlas via Mongoose 9. Exposes modular controllers executing atomic operations inside MongoDB transaction sessions. Incorporates rate limiting on authentication routes and Helmet security headers.
- **Frontend (React 18 & Vite 5):** Features an interactive graph visualizer built with React Flow (`@xyflow/react`) and styled with Tailwind CSS. Employs Dagre to calculate hierarchical graph layouts dynamically in either Left-to-Right (LR) or Top-to-Bottom (TB) orientations.

5.2 KEY FUNCTIONS (CODE EXPLANATION)

5.2.1 DAG Cycle Prevention Engine (Breadth-First Search)

To ensure biological validity, cellular lineage must form a strict Directed Acyclic Graph. The function `assertDagInvariant` evaluates proposed connections ($A \to B$) using Breadth-First Search traversal to verify that the parent is not reachable from the child:

$$\text{ValidEdge}(u, v) \iff u \notin \text{Reach}(v, G) \tag{5.1}$$

```
// backend/utils/dagGuard.js
async function isReachable(startId, targetId, session = null) {
  const startStr = startId.toString();
  const targetStr = targetId.toString();
  if (startStr === targetStr) return true;

  const visited = new Set([startStr]);
  const queue = [startId];
  const MAX_HOPS = 100;
  let hops = 0;

  while (queue.length > 0 && hops < MAX_HOPS) {
```

```
    hops++;
    const current = queue.shift();
    let query = LineageEdge.find({ sourceMaterialId: current });
    if (session) query = query.session(session);
    const edges = await query.lean();

    for (const edge of edges) {
      const nextStr = edge.targetMaterialId?.toString();
      if (!nextStr) continue;
      if (nextStr === targetStr) return true;
      if (!visited.has(nextStr)) {
        visited.add(nextStr);
        queue.push(edge.targetMaterialId);
      }
    }
  }
  return false;
}

async function assertDagInvariant(sourceId, targetId, session = null)
    {
  if (sourceId.toString() === targetId.toString()) {
    return { valid: false, message: 'Self-referential edges (A -> A)
   are strictly prohibited.' };
  }
  const cycleDetected = await isReachable(targetId, sourceId, session
   );
  if (cycleDetected) {
    return { valid: false, message: 'Cycle detected: Lineage must
   remain an acyclic DAG.' };
  }
  return { valid: true };
}
```

Listing 5.1: BFS Directed Acyclic Graph Cycle Guard Implementation

5.2.2 Institutional Compliance Gatekeeper

The compliance gate verifies that donor consent remains active and that the associated IRB approval is valid before permitting cellular derivations:

```
// backend/utils/complianceGate.js
async function validateComplianceForDerivation(donorId, approvalId,
   session = null) {
  let donorQuery = Donor.findById(donorId);
  if (session) donorQuery = donorQuery.session(session);
  const donor = await donorQuery;
```

```
  if (!donor) return { allowed: false, status: 404, message: 'Donor not found.' };

  const latestConsent = donor.consentHistory?.length > 0
    ? donor.consentHistory[donor.consentHistory.length - 1]
    : null;
  const isGiven = latestConsent ? latestConsent.status === 'GIVEN' : donor.consent?.status === 'Valid';
  if (!isGiven) {
    return { allowed: false, status: 409, message: 'Compliance violation: Donor consent WITHDRAWN.' };
  }

  let approvalQuery = EthicalApproval.findById(approvalId);
  if (session) approvalQuery = approvalQuery.session(session);
  const approval = await approvalQuery;
  if (!approval) return { allowed: false, status: 404, message: 'IRB approval not found.' };

  if (approval.status !== 'APPROVED') {
    return { allowed: false, status: 409, message: 'IRB Protocol is not APPROVED.' };
  }
  if (new Date(approval.validUntil) < new Date()) {
    return { allowed: false, status: 409, message: 'Ethical approval protocol has EXPIRED.' };
  }
  return { allowed: true, donor, approval };
}
```

Listing 5.2: Compliance Gate Middleware Logic

5.2.3 Atomic Derivation Workflows (Multi-Document Transactions)

Culture passaging is executed within a MongoDB ACID transaction. If cycle validation fails or an error occurs during edge insertion, all database operations roll back cleanly:

```
// backend/controllers/materialController.js
const passageCulture = async (req, res) => {
  const { id } = req.params;
  const { label, notes, storageLocation } = req.body;
  const parent = await findMaterial(id);
  if (!parent) return res.status(404).json({ message: 'Parent material not found.' });

  // 1. Verify Institutional Compliance
```

```
  const compliance = await validateComplianceForDerivation(parent.donorId, parent.approvalId);
  if (!compliance.allowed) {
    return res.status(compliance.status || 409).json({ message: compliance.message });
  }

  const nextPassage = (parent.passageNumber || parent.currentPassage || 0) + 1;
  const childMaterialId = label?.trim() || 'CLT-P' + nextPassage + '-' + Date.now().toString().slice(-4);

  // 2. Execute Atomic Multi-Document Transaction
  const session = await mongoose.startSession();
  try {
    session.startTransaction();
    const [child] = await BiologicalMaterial.create([{
      materialId: childMaterialId,
      materialType: 'Culture',
      passageNumber: nextPassage,
      currentPassage: nextPassage,
      storageLocation: storageLocation || 'Incubator 2 - Shelf B',
      status: 'ACTIVE',
      donorId: parent.donorId,
      approvalId: parent.approvalId,
      createdBy: req.user._id
    }], { session });

    // 3. Mathematical Cycle Verification
    const dagCheck = await assertDagInvariant(parent._id, child._id, session);
    if (!dagCheck.valid) {
      const err = new Error(dagCheck.message);
      err.statusCode = 409;
      throw err;
    }

    // 4. Create Directed Lineage Edge & Audit Entry
    const [edge] = await LineageEdge.create([{
      sourceMaterialId: parent._id,
      targetMaterialId: child._id,
      eventType: 'PASSAGED',
      notes: notes || 'Passaged to P' + nextPassage
    }], { session });

    await AuditLog.create([{
      userId: req.user._id,
```

```
      role: req.user.role,
      action: 'MATERIAL_PASSAGED',
      entityType: 'BiologicalMaterial',
      entityId: child._id,
      details: { parentId: parent.materialId, childId: child.materialId, passageNumber: nextPassage }
    }], { session });

    await session.commitTransaction();
    return res.status(201).json({ message: 'Passage created.', material: child, edge });
  } catch (err) {
    await session.abortTransaction();
    throw err;
  } finally {
    session.endSession();
  }
};
```

Listing 5.3: Atomic Multi-Document Passage Derivation Controller

5.2.4 Hierarchical Graph Layout Generation (Dagre Integration)

The frontend calculates 2D node coordinates and edge attachment points using Dagre to generate clean hierarchical lineage representations:

```
// frontend/src/utils/layoutGraph.js
export function getLayoutedElements(nodes, edges, direction = 'LR') {
  if (!nodes || nodes.length === 0) return { nodes: [], edges: [] };

  const dagreGraph = new dagre.graphlib.Graph();
  dagreGraph.setDefaultEdgeLabel(() => ({}));
  dagreGraph.setGraph({
    rankdir: direction, // 'LR' for Left-to-Right, 'TB' for Top-to-Bottom
    nodesep: 50,
    ranksep: 90
  });

  nodes.forEach((node) => {
    dagreGraph.setNode(node.id, { width: 250, height: 130 });
  });

  edges.forEach((edge) => {
    dagreGraph.setEdge(edge.source, edge.target);
  });

```

```
  dagre.layout(dagreGraph);

  const layoutedNodes = nodes.map((node) => {
    const pos = dagreGraph.node(node.id);
    return {
      ...node,
      targetPosition: direction === 'LR' ? 'left' : 'top',
      sourcePosition: direction === 'LR' ? 'right' : 'bottom',
      position: { x: pos.x - 125, y: pos.y - 65 }
    };
  });

  return { nodes: layoutedNodes, edges };
}
```

Listing 5.4: Hierarchical Lineage Graph Auto-Layout Function

CHAPTER 6

CONCLUSION & FUTURE WORK

6.1 CONCLUSION

The **StemTrack** platform delivers a comprehensive biological lineage tracking and ethical governance solution tailored specifically to human stem cell research. By replacing static database updates with an event-sourced Directed Acyclic Graph (DAG), the platform resolves key limitations of traditional Laboratory Information Management Systems (LIMS).

Key achievements of this project include:

- **Biological Provenance:** Derivations, splits, passages, and cryopreservations are modeled as immutable directed edges, establishing complete cellular genealogies.
- **Mathematical Integrity:** The Breadth-First Search DAG cycle prevention engine guarantees that biological lineages remain strictly acyclic, rejecting circular dependencies.
- **Integrated Ethical Governance:** Linking derivations directly to active IRB protocols and append-only donor consent records ensures that consent revocations propagate automatically across downstream experiments.
- **ACID Reliability:** MongoDB multi-document transactions guarantee that parent-child state changes, edges, and audit records update atomically.
- **Intuitive Visualization:** An interactive React Flow canvas paired with a 10×10 cryogenic slot matrix provides researchers with clear insight into both cellular lineages and physical freezer capacity.

6.2 FUTURE ENHANCEMENTS

Future development could expand StemTrack across several dimensions:

1. **Genomic Characterization Integration:** Link biological lineage nodes directly to Next-Generation Sequencing (NGS) data, single-cell RNA-seq files, and karyotype quality reports.

2. **Physical Barcode & RFID Scanner Support:** Integrate 2D DataMatrix barcode scanning for robotic cryogenic storage systems (e.g., Brooks Life Sciences, Hamilton BioOS).

3. **Blockchain-Backed Consent Verification:** Record donor consent milestones on a private distributed ledger (e.g., Hyperledger Fabric) to provide tamper-proof provenance for regulatory review.

4. **Automated Cell Growth Kinetics:** Integrate computer vision models to evaluate microscope images, automatically calculating cell confluence and recommending passaging schedules.

APPENDIX: SOURCE CODE

1. Project Folder Structure

```
stemcellproject/
|-- backend/
|   |-- controllers/
|   |   |-- approvalController.js
|   |   |-- auditController.js
|   |   |-- dashboardController.js
|   |   |-- donorController.js
|   |   |-- experimentController.js
|   |   |-- exportController.js
|   |   |-- lineageController.js
|   |   |-- materialController.js
|   |   `-- storageController.js
|   |-- middleware/
|   |   |-- auth.js
|   |   `-- validateObjectId.js
|   |-- models/
|   |   |-- AuditLog.js
|   |   |-- BiologicalMaterial.js
|   |   |-- Donor.js
|   |   |-- EthicalApproval.js
|   |   |-- Experiment.js
|   |   |-- LineageEdge.js
|   |   `-- User.js
|   |-- routes/
|   |-- utils/
|   |   |-- complianceGate.js
|   |   `-- dagGuard.js
|   |-- .env.example
|   |-- package.json
|   |-- seed.js
|   `-- server.js
`-- frontend/
    |-- src/
    |   |-- components/
    |   |-- context/
    |   |-- pages/
    |   `-- utils/
    `-- package.json
```

Listing 6.1: StemTrack Repository Directory Structure

2. Backend Package Installation Commands

```
# Root dependencies & cross-package orchestrator
npm install

# Backend dependencies installation
cd backend
npm install express mongoose dotenv cors helmet express-rate-limit
   express-validator bcryptjs jsonwebtoken
npm install --save-dev jest supertest

# Frontend dependencies installation
cd ../frontend
npm install react react-dom react-router-dom @xyflow/react dagre
   lucide-react tailwindcss @tailwindcss/vite
```

Listing 6.2: Package Installation Commands

3. Database Connection (`backend/config/db.js`)

```
// backend/server.js
const mongoose = require('mongoose');
const mongoURI = process.env.MONGODB_URI;

if (!mongoURI) {
  console.error('FAIL: MONGODB_URI environment variable is not
   defined.');
  process.exit(1);
}

mongoose.connect(mongoURI)
  .then(() => {
    console.log('SUCCESS: MongoDB Atlas Connected Successfully');
  })
  .catch((err) => {
    console.error('FAIL MongoDB Connection:', err.message || err);
    process.exit(1);
  });
```

Listing 6.3: Database Connection Script

4. Environment Variables (`backend/.env`)

```
# backend/.env.example
PORT=5000
MONGODB_URI=mongodb+srv://admin_user:SecurePass2026@cluster0.mongodb.
    net/stemtrack_db?retryWrites=true&w=majority
CLIENT_URL=http://localhost:5173
JWT_SECRET=stemtrack_super_secure_jwt_secret_key_2026
```

Listing 6.4: Environment Variables Configuration

5. User Model (`backend/models/User.js`)

```
// backend/models/User.js
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true, minlength: 2 },
    email: { type: String, required: true, unique: true, lowercase: true, trim: true },
    passwordHash: { type: String, required: true, select: false },
    role: { type: String, enum: ['admin', 'researcher', 'donor'], default: 'researcher' }
  },
  { timestamps: true }
);

userSchema.methods.toJSON = function () {
  const obj = this.toObject();
  delete obj.passwordHash;
  return obj;
};

module.exports = mongoose.model('User', userSchema);
```

Listing 6.5: User Mongoose Schema

6. Password Hashing and Registration

```
// backend/routes/authRoutes.js (Registration Handler)
router.post('/register', async (req, res) => {
  const { name, email, password, role } = req.body;
  if (!name || !email || !password) {
    return res.status(400).json({ message: 'Name, email and password are required.' });
  }
  if (password.length < 6) {
    return res.status(400).json({ message: 'Password must be at least 6 characters.' });
  }

  const safeRole = ['researcher', 'donor'].includes(role) ? role : 'researcher';
  try {
    const existing = await User.findOne({ email: email.toLowerCase().trim() });
    if (existing) {
      return res.status(409).json({ message: 'An account with this email already exists.' });
    }

    const passwordHash = await bcrypt.hash(password, 12);
    const user = await User.create({ name, email, passwordHash, role: safeRole });
    const token = signToken(user);

    res.status(201).json({ message: 'Account created successfully.', token, user });
  } catch (err) {
    res.status(500).json({ message: 'Internal server error during registration.' });
  }
});
```

Listing 6.6: Password Hashing and User Registration Handler

7. Login and JWT Generation

```
// backend/routes/authRoutes.js (Login Handler)
function signToken(user) {
  return jwt.sign(
    { sub: user._id, email: user.email, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: '1h' }
  );
}

router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password) {
    return res.status(400).json({ message: 'Email and password are required.' });
  }

  try {
    const user = await User.findOne({ email: email.toLowerCase().trim() }).select('+passwordHash');
    if (!user) return res.status(401).json({ message: 'Invalid email or password.' });

    const isMatch = await bcrypt.compare(password, user.passwordHash);
    if (!isMatch) return res.status(401).json({ message: 'Invalid email or password.' });

    const token = signToken(user);
    res.status(200).json({ message: 'Login successful.', token, user: user.toJSON() });
  } catch (err) {
    res.status(500).json({ message: 'Internal server error during login.' });
  }
});
```

Listing 6.7: User Login and JWT Generation

8. Authentication Middleware

```
// backend/middleware/auth.js
const jwt = require('jsonwebtoken');
const User = require('../models/User');

async function requireAuth(req, res, next) {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ message: 'Authentication required. Please provide a Bearer token.' });
  }
  const token = authHeader.split(' ')[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.sub);
    if (!user) return res.status(401).json({ message: 'User account no longer exists.' });
    req.user = user;
    next();
  } catch (err) {
    return res.status(401).json({ message: err.name === 'TokenExpiredError' ? 'Token expired.' : 'Invalid token.' });
  }
}

function requireRole(...roles) {
  return (req, res, next) => {
    if (!req.user) return res.status(401).json({ message: 'Not authenticated.' });
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ message: 'Access denied. Required: ' + roles.join(' or ') });
    }
    next();
  };
}
```

Listing 6.8: Authentication and Role Verification Middleware

9. Create API (Sample Registration)

```
// backend/controllers/materialController.js
const registerSample = async (req, res) => {
  const { donorId, approvalId, label, storageLocation } = req.body;
  if (!donorId || !approvalId || !label) {
    return res.status(400).json({ message: 'donorId, approvalId, and label are required.' });
  }

  // Pre-validate compliance
  const compliance = await validateComplianceForDerivation(donorId, approvalId);
  if (!compliance.allowed) {
    return res.status(compliance.status || 409).json({ message: compliance.message });
  }

  const existing = await BiologicalMaterial.findOne({ materialId: label.trim() });
  if (existing) return res.status(409).json({ message: 'Material ID already exists.' });

  const session = await mongoose.startSession();
  try {
    session.startTransaction();
    const [sample] = await BiologicalMaterial.create([{
      materialId: label.trim(),
      materialType: 'Sample',
      passageNumber: 0,
      currentPassage: 0,
      storageLocation: storageLocation || 'Primary Cold Room R-01',
      status: 'ACTIVE',
      donorId: compliance.donor._id,
      approvalId: compliance.approval._id,
      createdBy: req.user._id
    }], { session });

    await AuditLog.create([{
      userId: req.user._id,
      role: req.user.role,
      action: 'SAMPLE_REGISTERED',
      entityType: 'BiologicalMaterial',
      entityId: sample._id,
      details: { materialId: sample.materialId, donorId: compliance.donor.donorId }
    }], { session });
```

```
40
41      await session.commitTransaction();
42      return res.status(201).json({ message: 'Intake sample registered
        successfully.', material: sample });
43    } catch (err) {
44      await session.abortTransaction();
45      throw err;
46    } finally {
47      session.endSession();
48    }
49  };
```

Listing 6.9: Intake Sample Registration Endpoint Controller

10. Read API (Lineage Graph BFS Traversal)

```
// backend/controllers/lineageController.js
const getLineageByMaterialId = async (req, res) => {
  const rawId = req.params.id;
  const isObjectId = /^[0-9a-fA-F]{24}$/.test(rawId);
  const starting = await BiologicalMaterial.findOne(
    isObjectId ? { $or: [{ _id: rawId }, { materialId: rawId }] } : { materialId: rawId }
  ).populate('donorId');

  if (!starting) return res.status(404).json({ message: 'Biological material not found' });

  // Bidirectional BFS traversal
  const visitedNodeIds = new Set([starting._id.toString()]);
  const edgesMap = new Map();
  let frontier = [starting._id];
  let depth = 0;

  while (frontier.length > 0 && depth < 50) {
    depth++;
    const connectedEdges = await LineageEdge.find({
      $or: [{ sourceMaterialId: { $in: frontier } }, { targetMaterialId: { $in: frontier } }]
    }).lean();

    const nextFrontier = [];
    for (const edge of connectedEdges) {
      edgesMap.set(edge._id.toString(), edge);
      const sId = edge.sourceMaterialId.toString();
      const tId = edge.targetMaterialId.toString();
      if (!visitedNodeIds.has(sId)) { visitedNodeIds.add(sId); nextFrontier.push(edge.sourceMaterialId); }
      if (!visitedNodeIds.has(tId)) { visitedNodeIds.add(tId); nextFrontier.push(edge.targetMaterialId); }
    }
    frontier = nextFrontier;
  }

  const materials = await BiologicalMaterial.find({ _id: { $in: Array.from(visitedNodeIds) } })
    .populate('donorId').populate('approvalId').lean();

  const nodes = materials.map((mat) => ({
    id: mat._id.toString(),
    type: mat.materialType,
```

```
    position: { x: 0, y: 0 },
    data: {
      label: mat.materialId,
      materialId: mat.materialId,
      materialType: mat.materialType,
      currentPassage: mat.passageNumber ?? mat.currentPassage,
      status: mat.status
    }
  }));

  const edges = Array.from(edgesMap.values()).map((edge) => ({
    id: edge._id.toString(),
    source: edge.sourceMaterialId.toString(),
    target: edge.targetMaterialId.toString(),
    label: edge.eventType
  }));

  return res.status(200).json({ nodes, edges });
};
```

Listing 6.10: Lineage Graph Bidirectional BFS Traversal Controller

11. Update API (Cryopreservation Slot Check)

```
// backend/controllers/materialController.js
const cryopreserveMaterial = async (req, res) => {
  const { id } = req.params;
  const { freezer, box, position } = req.body;
  if (!freezer || !box || !position) {
    return res.status(400).json({ message: 'Freezer, box, and position are required.' });
  }

  const material = await findMaterial(id);
  if (!material) return res.status(404).json({ message: 'Material not found.' });

  // Verify storage position vacancy
  const occupied = await BiologicalMaterial.findOne({
    _id: { $ne: material._id },
    status: 'STORED',
    'storageLocation.freezer': freezer.trim(),
    'storageLocation.box': box.trim(),
    'storageLocation.position': position.trim().toUpperCase()
  });

  if (occupied) {
    return res.status(409).json({ message: 'Slot ' + freezer + '>' + box + '>' + position + ' occupied by ' + occupied.materialId });
  }

  material.status = 'STORED';
  material.materialType = 'CryoVial';
  material.storageLocation = { freezer: freezer.trim(), box: box.trim(), position: position.trim().toUpperCase() };
  await material.save();

  return res.status(200).json({ message: 'Material stored in cryo repository.', material });
};
```

Listing 6.11: Cryopreservation Slot Vacancy Check and Allocation Controller

12. Delete API (Discard Contaminated Material)

```
// backend/controllers/materialController.js
const discardMaterial = async (req, res) => {
  const { id } = req.params;
  const { reason } = req.body;
  if (!reason || !reason.trim()) {
    return res.status(400).json({ message: 'Reason is required for biological discard.' });
  }

  const material = await findMaterial(id);
  if (!material) return res.status(404).json({ message: 'Material not found.' });
  if (material.status === 'DISCARDED') {
    return res.status(409).json({ message: 'Material has already been discarded.' });
  }

  material.status = 'DISCARDED';
  await material.save();

  await AuditLog.create({
    userId: req.user._id,
    role: req.user.role,
    action: 'MATERIAL_DISCARDED',
    entityType: 'BiologicalMaterial',
    entityId: material._id,
    details: { materialId: material.materialId, reason: reason.trim() }
  });

  return res.status(200).json({ message: 'Material securely discarded.', material });
};
```

Listing 6.12: Biological Material Discard and Audit Trail Entry

13. Express Routes

```
// backend/routes/materialRoutes.js
const express = require('express');
const router = express.Router();
const {
  registerSample, deriveCulture, passageCulture, splitCulture,
  cryopreserveMaterial, retrieveMaterial, useMaterialInExperiment,
  discardMaterial, listMaterials, getMaterialById
} = require('../controllers/materialController');
const { requireRole } = require('../middleware/auth');
const { validateObjectId } = require('../middleware/validateObjectId');

router.get('/', listMaterials);
router.get('/:id', validateObjectId('id'), getMaterialById);
router.post('/sample', requireRole('researcher', 'admin'), registerSample);
router.post('/:id/culture', requireRole('researcher', 'admin'), validateObjectId('id'), deriveCulture);
router.post('/:id/passage', requireRole('researcher', 'admin'), validateObjectId('id'), passageCulture);
router.post('/:id/split', requireRole('researcher', 'admin'), validateObjectId('id'), splitCulture);
router.post('/:id/cryopreserve', requireRole('researcher', 'admin'), validateObjectId('id'), cryopreserveMaterial);
router.post('/:id/retrieve', requireRole('researcher', 'admin'), validateObjectId('id'), retrieveMaterial);
router.post('/:id/use', requireRole('researcher', 'admin'), validateObjectId('id'), useMaterialInExperiment);
router.post('/:id/discard', requireRole('researcher', 'admin'), validateObjectId('id'), discardMaterial);

module.exports = router;
```

Listing 6.13: Biological Material Express Routes Configuration

14. Main Server File

```
// backend/server.js
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');
const { requireAuth } = require('./middleware/auth');

const app = express();
app.use(helmet({ contentSecurityPolicy: false }));
app.use(cors({ origin: true, credentials: true }));
app.use(express.json());

// Routes
app.use('/api/auth/login', rateLimit({ windowMs: 15 * 60 * 1000, max:
    50 }));
app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/lineage', requireAuth, require('./routes/lineageRoutes
    '));
app.use('/api/materials', requireAuth, require('./routes/
    materialRoutes'));
app.use('/api/donors', requireAuth, require('./routes/donorRoutes'));
app.use('/api/approvals', requireAuth, require('./routes/
    approvalRoutes'));
app.use('/api/experiments', requireAuth, require('./routes/
    experimentRoutes'));
app.use('/api/storage', requireAuth, require('./routes/storageRoutes
    '));
app.use('/api/dashboard', requireAuth, require('./routes/
    dashboardRoutes'));
app.use('/api/audit', requireAuth, require('./routes/auditRoutes'));
app.use('/api/export', requireAuth, require('./routes/exportRoutes'))
    ;

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log('StemTrack API Server running on
    port ' + PORT));
module.exports = app;
```

Listing 6.14: Express Application Entry Point and Security Configuration

15. React Login Component

```
// frontend/src/pages/LoginPage.jsx
import React, { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

export default function LoginPage() {
  const { login } = useAuth();
  const navigate = useNavigate();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      await login(email.trim(), password);
      navigate('/');
    } catch (err) {
      setError(err.message || 'Authentication failed');
    }
  };

  return (
    <div className="min-h-screen bg-[#090d16] flex items-center
    justify-center p-4">
      <form onSubmit={handleSubmit} className="bg-[#0d1322] border
    border-slate-800 p-8 rounded-2xl w-full max-w-md shadow-2xl">
        <h1 className="text-xl font-bold text-white mb-2">StemTrack
    Login</h1>
        {error && <div className="p-3 bg-red-950/60 border border-red
    -800 text-red-300 text-xs rounded mb-4">{error}</div>}
        <input type="email" placeholder="Email" value={email}
    onChange={e=>setEmail(e.target.value)} required
               className="w-full bg-slate-900 border border-slate-700
     p-2 text-white rounded mb-3 text-sm" />
        <input type="password" placeholder="Password" value={password
    } onChange={e=>setPassword(e.target.value)} required
               className="w-full bg-slate-900 border border-slate-700
     p-2 text-white rounded mb-4 text-sm" />
        <button type="submit" className="w-full py-2.5 bg-sky-600
    hover:bg-sky-500 text-white font-bold rounded">
          Sign In
        </button>
      </form>
    </div>
```

```
37     );
38 }
```

Listing 6.15: React User Authentication Login Interface

16. Admin Dashboard Logic

```
// backend/controllers/dashboardController.js
const getDashboardMetrics = async (req, res) => {
  const now = new Date();
  const in30Days = new Date(now.getTime() + 30 * 24 * 60 * 60 * 1000);

  const [donorCount, activeCount, storedCount, expCount, donors, expiring, recent] = await Promise.all([
    Donor.countDocuments({}),
    BiologicalMaterial.countDocuments({ status: { $in: ['ACTIVE', 'Active'] } }),
    BiologicalMaterial.countDocuments({ status: { $in: ['STORED', 'Cryopreserved'] } }),
    Experiment.countDocuments({}),
    Donor.find({}).select('consentHistory consent').lean(),
    EthicalApproval.find({ status: 'APPROVED', validUntil: { $gte: now, $lte: in30Days } }).lean(),
    AuditLog.find({}).populate('userId', 'name role').sort({ timestamp: -1 }).limit(10).lean()
  ]);

  let given = 0, withdrawn = 0;
  for (const d of donors) {
    const latest = d.consentHistory?.[d.consentHistory.length - 1];
    if ((latest?.status || d.consent?.status) === 'GIVEN' || (latest?.status || d.consent?.status) === 'Valid') given++;
    else withdrawn++;
  }

  return res.status(200).json({
    totals: { donors: donorCount, activeMaterials: activeCount, storedVials: storedCount, experiments: expCount },
    consent: { given, withdrawn, total: donorCount },
    expiringApprovals: expiring,
    recentActivity: recent
  });
};
```

Listing 6.16: Dashboard KPI Metrics and Regulatory Aggregation Controller

17. REST API Request and Response Examples

Authentication Endpoint

Request: `POST /api/auth/login`

```
{
  "email": "researcher@stemtrack.io",
  "password": "ResearchPass123!"
}
```

Listing 6.17: POST /api/auth/login Request Payload

Response: `HTTP 200 OK`

```
{
  "message": "Login successful.",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "_id": "67af8a1012fbc98765432101",
    "name": "Dr. Maya Lin (Researcher)",
    "email": "researcher@stemtrack.io",
    "role": "researcher"
  }
}
```

Listing 6.18: POST /api/auth/login Response Payload

Culture Derivation Endpoint

Request: `POST /api/materials/67af8a1012fbc98765432104/passage`

```
{
  "label": "CLT-P2-BRANCH-A",
  "notes": "Passaged onto geltrex feeder matrix",
  "storageLocation": "Incubator 3 - Shelf A"
}
```

Listing 6.19: POST /api/materials/:id/passage Request Payload

Response: `HTTP 201 Created`

```
{
  "message": "Passage P2 created successfully.",
  "material": {
    "_id": "67af8b2212fbc98765432199",
    "materialId": "CLT-P2-BRANCH-A",
    "materialType": "Culture",
    "passageNumber": 2,
    "currentPassage": 2,
```

```
    "status": "ACTIVE",
    "storageLocation": "Incubator 3 - Shelf A"
  },
  "edge": {
    "_id": "67af8b2212fbc98765432200",
    "sourceMaterialId": "67af8a1012fbc98765432104",
    "targetMaterialId": "67af8b2212fbc98765432199",
    "eventType": "PASSAGED"
  }
}
```

Listing 6.20: POST /api/materials/:id/passage Response Payload

Compliance Gate Rejection Example

Request: Derivation from sample associated with donor `DNR-SEED-002` (Withdrawn consent)

Response: `HTTP 409 Conflict`

```
{
  "message": "Compliance violation: Donor consent is currently
    WITHDRAWN or inactive. Derivation blocked."
}
```

Listing 6.21: Compliance Gate Rejection Response Payload

99

Takahashi, K., and Yamanaka, S. (2006). “Induction of pluripotent stem cells from mouse embryonic and adult fibroblast cultures by defined factors,” *Cell*, vol. 126, no. 4, pp. 663–676.

Isasi, R., and Knoppers, B. M. (2015). “Oversight of human stem cell research: Guidance for researchers and review committees,” *Stem Cell Reports*, vol. 4, no. 4, pp. 519–524.

Fowler, M. (2017). *Event Sourcing Patterns and Architectural Principles in Modern Software Engineering*, Addison-Wesley Professional, Boston.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms*, 4th ed., MIT Press, Cambridge, pp. 589–612.

National Institutes of Health (NIH). (2016). *NIH Guidelines for Human Stem Cell Research*, U.S. Department of Health and Human Services, Bethesda.

World Medical Association. (2013). “World Medical Association Declaration of Helsinki: Ethical principles for medical research involving human subjects,” *JAMA*, vol. 310, no. 20, pp. 2191–2194.

Bader, G. D., and Hogue, C. W. (2003). “An automated method for finding molecular complexes in large protein interaction graphs,” *BMC Bioinformatics*, vol. 4, no. 1, art. no. 2.

Banker, K., Bakkum, P., Verch, S., and Garrett, D. (2016). *MongoDB in Action: Covers MongoDB version 3.0+*, Manning Publications, Shelter Island.

Banks, A., and Porcello, E. (2020). *Learning React: Modern Patterns for Developing React Applications*, 2nd ed., O’Reilly Media, Sebastopol.

Shafranovich, Y. (2005). *Common Format and MIME Type for Comma-Separated Values (CSV) Files*, RFC 4180, Internet Engineering Task Force (IETF).

SCREENSHOTS

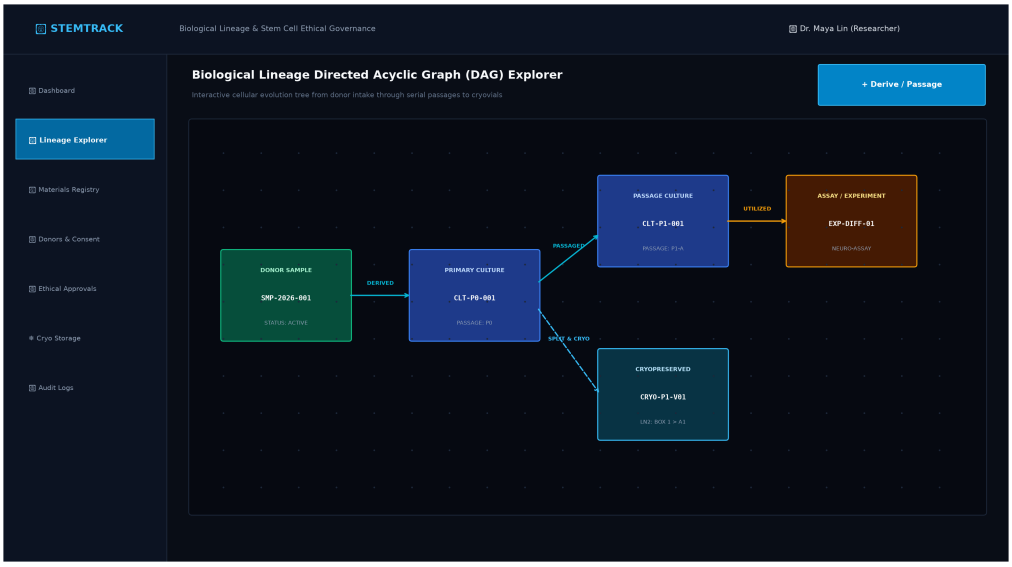


Figure 4: Biological Lineage Directed Acyclic Graph Explorer

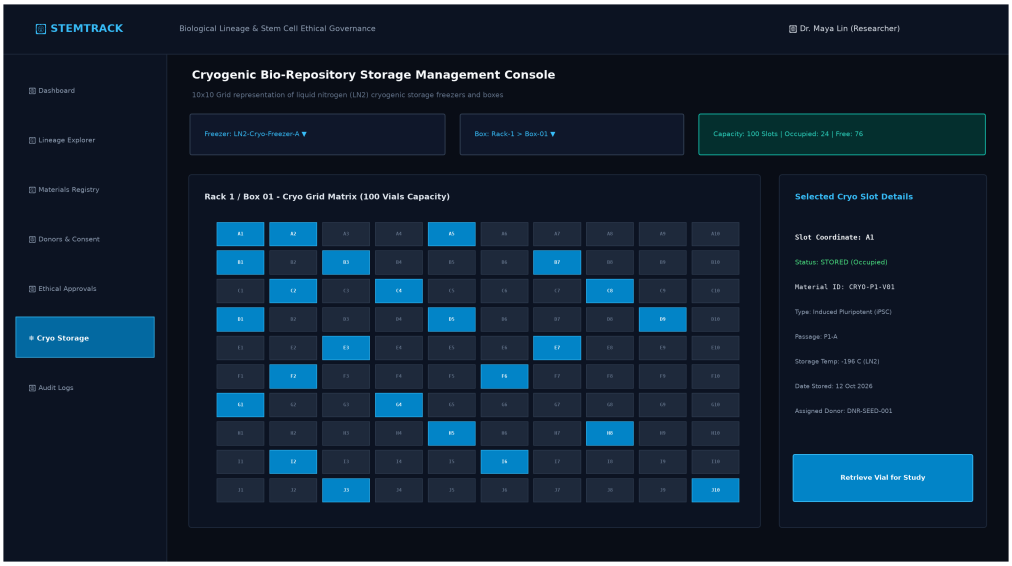


Figure 5: Cryogenic Bio-Repository Storage Management Console

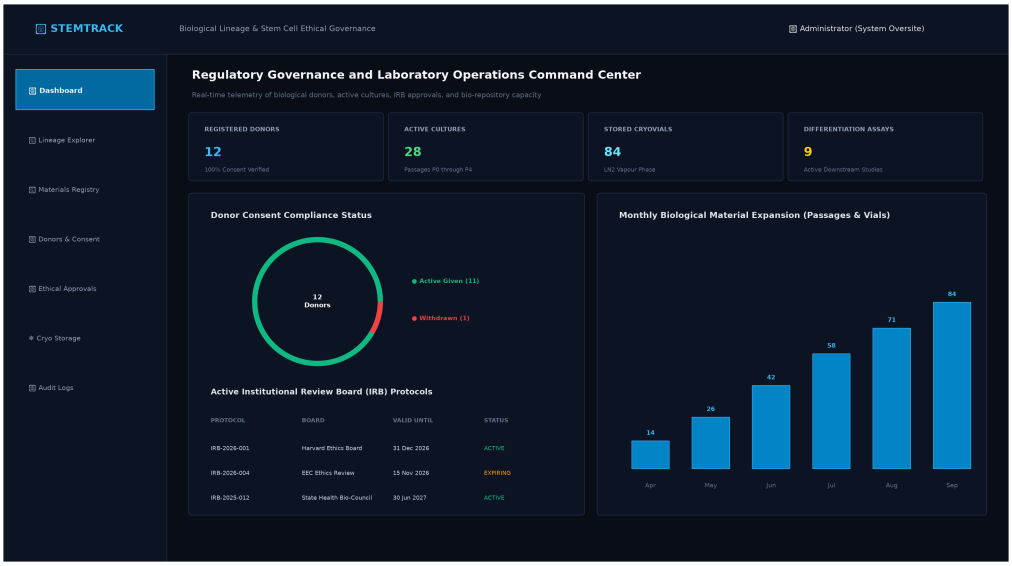


Figure 6: Regulatory Compliance and Operations Command Center

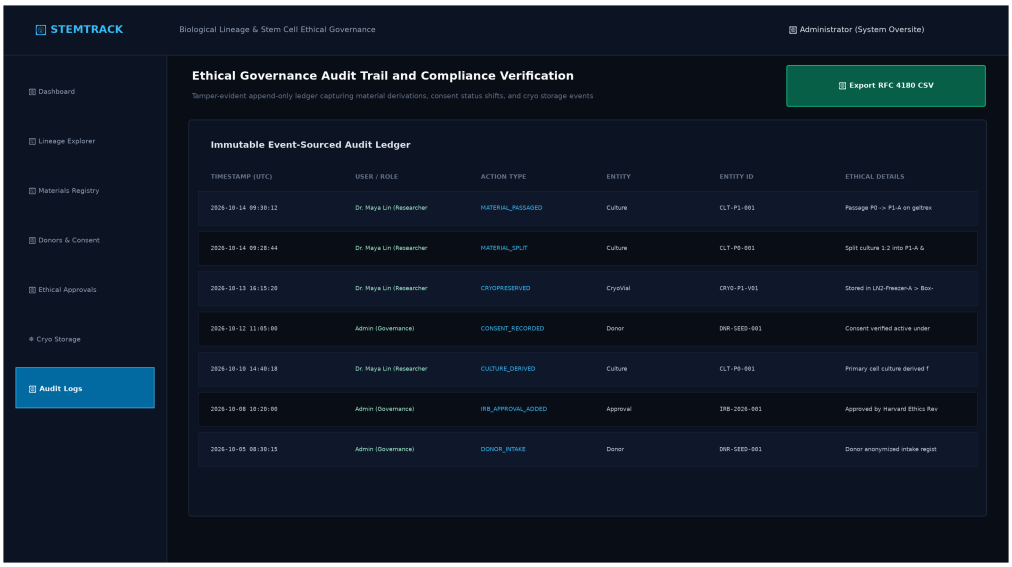


Figure 7: Ethical Governance and Immutable Audit Log Console